Table of Contents

Terraform

1. Introducción

1.1. Qué es terraform

Terraform es una herramienta de *Infraestructura como Código (IaC)* de código abierto, desarrollada por HashiCorp, que permite a los desarrolladores y equipos de operaciones definir, aprovisionar y gestionar recursos de infraestructura de manera automatizada.

A diferencia de la gestión manual en consolas de nube, Terraform utiliza un lenguaje declarativo llamado *HCL (HashiCorp Configuration Language)*, donde simplemente describes el estado final deseado de tu infraestructura (como servidores, redes o bases de datos) y la herramienta se encarga de calcular las diferencias y ejecutar los cambios necesarios para alcanzar ese estado.

1.2. Características Principales

Agnóstico de la nube
Funciona con múltiples proveedores como AWS, Azure, Google Cloud y servicios SaaS.

Gestión del Estado
Mantiene un archivo de registro (`terraform.tfstate`) que actúa como la "fuente de verdad" de lo que realmente existe en tu entorno.

Planificación de ejecución
El comando `terraform plan` te permite previsualizar los cambios antes de que se apliquen, evitando sorpresas desagradables.

Modularidad
Permite organizar el código en componentes reutilizables, facilitando la escalabilidad y la consistencia en grandes proyectos.

1.3. Proveedores

Los proveedores de Terraform son los complementos o plugins que actúan como puentes de comunicación entre el núcleo de Terraform y las diversas plataformas externas.

Dado que Terraform es una herramienta agnóstica, no sabe "hablar" de forma nativa con cada servicio; por ello, utiliza a los proveedores para traducir el código declarativo (HCL) en llamadas a la API específicas de cada servicio.

1.3.1. Tipos de Proveedores

Para facilitar la gestión, el ecosistema de Terraform los organiza en tres categorías principales dentro de su **Registry**:

Oficiales
Desarrollados y mantenidos directamente por HashiCorp.

Socios (Partner)
Publicados por empresas terceras que mantienen su propio hardware o software y colaboran con HashiCorp (ej. Cloudflare, Datadog, Oracle).

Comunidad
Creados por desarrolladores independientes para servicios específicos o nichos que no cuentan con soporte oficial.

1.3.2. Principales proveedores

Destacaré entre los principales proveedores (y otros que me han llamado la atención):

- AWS
- Azure
- Google Cloud Platform
- Kubernetes
- Oracle Cloud Infrastructure
- Digital Ocean
- Linode
- Ionos
- Heztner
- Proxmox
- Docker
- Ansible

1.4. Alternativas

OpenTofu surge como una alternativa directa y de código abierto a Terraform tras el cambio de licencia de HashiCorp en 2023, cuando pasaron de una licencia Mozilla (open source) a una *Business Source License (BUSL)*, lo que impuso restricciones comerciales al uso del software (recordemos que Hashicorp fue comprada por IBM).

Ante este cambio, la comunidad y varias empresas impulsaron OpenTofu bajo el paraguas de la *Linux Foundation* para garantizar que la herramienta de Infraestructura como Código (IaC) permanezca verdaderamente abierta, neutral y libre de intereses corporativos restrictivos.

Es, en esencia, un *fork* (una bifurcación) diseñado para ser un reemplazo (*drop-in replacement*), manteniendo la compatibilidad con el lenguaje HCL y los proveedores existentes, pero con un modelo de gobernanza comunitaria que asegura que el desarrollo futuro sea impulsado por las necesidades de los usuarios y no por estrategias de monetización privadas.

1.4.1. ¿Por qué elegir OpenTofu?

Licencia Permisiva
Utiliza la licencia *Apache 2.0*, lo que garantiza que siempre será gratuita y abierta para cualquier tipo de uso empresarial.

Compatibilidad Total (por ahora)
Puedes migrar proyectos de Terraform a OpenTofu de forma sencilla, ya que interpreta el mismo código y gestiona los mismos archivos de estado.

Innovación Comunitaria
Al ser un proyecto de la Linux Foundation, las nuevas funciones se deciden de forma transparente y pública, evitando el bloqueo de funcionalidades tras muros de pago.

Independencia de Proveedor
Elimina el riesgo de que una sola empresa cambie las reglas del juego de la noche a la mañana sobre una herramienta crítica para tu infraestructura.

2. Estructura

Un proyecto de Terraform bien organizado no requiere una estructura rígida por parte de la herramienta, pero la comunidad ha estandarizado una anatomía de archivos para mantener el código limpio, escalable y fácil de entender. En lugar de tener miles de líneas en un solo lugar, Terraform busca archivos con la extensión `.tf` que separan las responsabilidades: la configuración de los proveedores, la definición de los recursos, la declaración de variables y la salida de datos.

2.1. Archivos Fundamentales

`main.tf`
Aquí se definen los recursos principales que quieres crear (instancias, redes, etc.). En proyectos pequeños es el archivo principal, mientras que en grandes arquitecturas suele llamar a otros módulos.

`variables.tf`
Aquí se declaran las variables de entrada. Permite parametrizar tu código para que sea reutilizable (por ejemplo, definir el tipo de instancia o la región) sin tener que editar el código base.

`outputs.tf`
Define qué información quieres que Terraform te devuelva una vez terminada la ejecución, como la dirección IP pública de un servidor o la URL de un balanceador de carga.

`providers.tf`
Se utiliza para configurar los proveedores (AWS, Azure, etc.) y las versiones requeridas, separando la lógica de conexión de la infraestructura misma.

`terraform.tfvars`
Este archivo contiene los valores reales de las variables definidas en `variables.tf`. Es el lugar donde pones los datos específicos para un entorno concreto (dev, staging, prod).

2.2. Archivos Generados (No editables)

`.terraform/`
Una carpeta oculta que se crea al ejecutar `terraform init`. Contiene los binarios de los proveedores y módulos descargados. **No se sube a Github**.

`terraform.tfstate`
Es el archivo más crítico; es un JSON que mapea tus archivos de configuración con los recursos reales creados en la nube. **Nunca debe editarse manualmente**.

2.3. `.gitignore`

Como siempre es importante tener un fichero `.gitignore` que nos proteja de subir ficheros indebidos a Git. Fíjate que ignoramos el `.terraform`, el `.tfstate` y los `.tfvars`.

```
# Backup files
*~

# Local .terraform directories
.terraform/

# .tfstate files
*.tfstate
*.tfstate.*

# Crash log files
crash.log
crash.*.log

# Exclude all .tfvars files, which are likely to contain sensitive data, such as
# password, private keys, and other secrets. These should not be part of version
# control as they are data points which are potentially sensitive and subject
# to change depending on the environment.
*.tfvars
*.tfvars.json

# Ignore override files as they are usually used to override resources locally and so
# are not checked in
override.tf
override.tf.json
*_override.tf
*_override.tf.json

# Ignore transient lock info files created by terraform apply
.terraform.tfstate.lock.info
.terraform.lock.hcl

# Include override files you do wish to add to version control using negated pattern
# !example_override.tf

# Include tfplan files to ignore the plan output of command: terraform plan -out=tfplan
# example: *tfplan*

# Ignore CLI configuration files
.terraformrc
terraform.rc

# Optional: ignore graph output files generated by `terraform graph`
# *.dot

# Optional: ignore plan files saved before destroying Terraform configuration
# Uncomment the line below if you want to ignore planout files.
# planout
```

2.4. `providers.tf`

Este fichero sería un ejemplo de `providers.tf` para utilizar el proveedor de Proxmox y desplegar en el instituto.

```
terraform {
  required_version = ">= 1.0.0"

  required_providers {
    proxmox = {
      source  = "telmate/proxmox"
      version = "3.0.1-rc07"
    }
  }
}

provider "proxmox" {
  # URL de la API (ej: https://192.168.1.100:8006/api2/json)
  pm_api_url = var.pm_api_url

  # Autenticación mediante API Token (Recomendado por seguridad)
  pm_api_token_id     = var.pm_api_token_id
  pm_api_token_secret = var.pm_api_token_secret

  # Configuración de TLS
  # Se pone en 'true' si usas certificados auto-firmados (sin SSL válido)
  pm_tls_insecure = true
}
```

2.5. `main.tf`

A continuación pongo un ejemplo de fichero `main.tf` preparado para desplegar una máquina virtual hecha a partir de una plantilla. Los valores más importantes se pasan como variables.

```
resource "proxmox_vm_qemu" "vm_instance_1" {

  # Nombre de la MV
  name = "ciber-fraya-vm-1"

  # Nombre de tu nodo en Proxmox
  target_node = var.pm_node

  # Habilita el agente de qemu
  agent = 1

  # Memoria de la MV
  memory = 1024

  # Tipo de controlador SCSI
  scsihw = "virtio-scsi-single"

  # Tipo de sistema operativo
  os_type = "cloud-init"

  # Pool al que pertenece la MV
  pool = var.pm_pool

  # Estado de la MV
  vm_state = "running"

  # El orden de arranque debe ser el mismo que el del disco del SO de la plantilla
  boot = "order=scsi0"

  # Plantilla de la que se clona la MV, que ya debe estar creada en Proxmox
  clone = var.pm_template

  # Clave (pública) SSH
  sshkeys = file(var.ssh_key_file)

  #
  # Cloud-Init configuration
  #

  # Habilita el agente de qemu (/var/lib/vz/snippets/qemu-guest-agent.yml)
  cicustom    = "vendor=local:snippets/qemu-guest-agent.yml"

  # Hace apt update & apt upgrade al arrancar la VM
  ciupgrade  = true

  # DNS
  # nameserver = "192.168.0.1 8.8.8.8"

  # Configura IP
  # ipconfig0  = "ip=192.168.0.200/24,gw=192.168.0.1,ip6=dhcp"
  ipconfig0  = "ip=dhcp,ip6=dhcp"

  # Deshabilita IPv6
  skip_ipv6  = true

  # Usuario y contraseña de la MV
  ciuser     = var.ci_user
  cipassword = var.ci_password

  # La mayoría de las imágenes cloud-init requieren un dispositivo
  # serie para su visualización
  serial { id = 0 }

  # Configuración de hardware
  cpu {

    # Número de núcleos que la MV verá
    cores = 2

    # Número de sockets que la MV verá
    sockets = 1

    # El número total de hebras o vCPUs será cores * sockets
    # Tipo de modelo de CPU virtual
    type = "host"
  }

  # Configuración de disco

  disks {
    scsi {
      scsi0 {
        disk {
          storage = var.pm_storage
          size = "3G"
        }
      }
    }
    ide {
      # Algunas imágenes requieren un disco cloud-init en el controlador IDE,
      # otras en el controlador SCSI o SATA
      ide1 {
        cloudinit {
          storage = var.pm_storage
        }
      }
    }
  }

  # Configuración de red

  network {
    id     = 0
    model  = "virtio"
    bridge = var.pm_bridge
  }

}
```

2.5.1. Cloud image

La plantilla tiene la particularidad de ser una *cloud image* (o imagen de nube) que es, en esencia, una plantilla de sistema operativo preconfigurada y optimizada para ejecutarse en entornos virtualizados o de nube.

A diferencia de una instalación tradicional donde descargas un archivo ISO y sigues un asistente paso a paso, una Cloud Image es un disco duro virtual ya *listo para usar*. Cuando instalas un sistema mediante ISO, el proceso es manual y lento. Las Cloud Images están diseñadas para la automatización y el escalado rápido. Sus características principales son:

Sin instalador
No hay menús de "Siguiente, Siguiente". El disco ya contiene el sistema operativo instalado.

Minimalismo
Suelen venir sin entorno gráfico y con lo mínimo indispensable para funcionar, no tienen drivers para muchas cosas, lo que las hace ligeras.

Cloud-Init
Es un paquete que permite configurar la máquina (usuario, contraseña, llaves SSH, red) en el primer arranque sin tener que entrar manualmente.

2.5.2. Qemu Gest Agent

En el contexto de Proxmox nuestra imagen tiene el *QEMU Guest Agent* (qemu-guest-agent) qué es un proceso (o demonio) que se ejecuta dentro de la máquina virtual y actúa como un puente de comunicación directa entre el sistema operativo invitado (la Cloud Image) y el hipervisor (Proxmox).

Este componente es vital para que la gestión de la VM sea "inteligente" y no una caja negra para el servidor.

Cuando instalas y activas el agente en una Cloud Image, Proxmox puede realizar operaciones sobre esa máquina:

Apagado y Reinicio Limpio
Sin el agente, Proxmox envía una señal de energía (como pulsar el botón de la torre). Con el agente, Proxmox le ordena al sistema operativo que cierre los procesos correctamente, evitando corrupción de datos.

Información de Red Real
Permite que Proxmox muestre la dirección IP interna de la VM directamente en el panel de resumen, sin que tengas que entrar a la consola a mirarla.

Congelación del Sistema de Archivos (FS Freeze)
Permite hacer Backups y Snapshots. El agente pausa las escrituras en disco por un milisegundo para asegurar que la copia de seguridad sea consistente y no haya archivos corruptos.

Sincronización de Hora
Asegura que el reloj de la VM sea exacto al del servidor físico

Le he anadido la ejecución de un fichero que se ejecuta al iniciar la VM *la primera vez* llamado `/var/lib/vz/snippets/qemu-guest-agent.yml` y que además de habilitar al agente de qemu podría instalar lo que quisiéramos (aunque lo mejor sería provisionar con otra herramienta como Ansible).

```
#cloud-config
runcmd:
  - apt update
  - apt install -y qemu-guest-agent
  - systemctl enable --now qemu-guest-agent
```

Pues vaya rollo, y si no lo uso? Se verá un mensaje *No guest agent configured*, las copias de seguridad podrían fallar al intentar congelar el disco y si le das a *shutdown*, Proxmox espera un tiempo y si la VM no responde la apaga por las bravas.

2.6. `variables.tf`

Para que el archivo anterior funcione, debes definir las variables que estás invocando. Esto permite que Terraform sepa qué tipo de datos esperar.

```
# Este archivo define las variables de entrada para la configuración de Terraform,
# permitiendo la parametrización de credenciales y configuraciones del despliegue.

variable "pm_api_url" {
  description = "URL del servidor Proxmox"
  type        = string
}

variable "pm_node" {
  description = "Proxmox target node"
  type        = string
}

variable "pm_template" {
  description = "Plantilla de la que se clona la MV"
  type = string
}

variable "pm_user" {
  description = "PVE username"
  type        = string
}

variable "pm_password" {
  description = "PVE password"
  type        = string
}

variable "pm_api_token_id" {
  description = "Id del token"
  type        = string
}

variable "pm_api_token_secret" {
 description = "Token secret"
 type        = string
 sensitive   = true
}

variable "pm_pool" {
  description = "Pool del usuario"
  type = string
}

variable "pm_storage" {
  description = "Almacenamiento de los discos"
  type = string
}

variable "pm_bridge" {
  description = "Tarjeta de red de puente"
  type = string
}

variable "ci_user" {
  description = "Usuario por defecto para la máquina virtual (Cloud-Init)"
  type        = string
}

variable "ci_password" {
  description = "Contraseña para el usuario de la máquina virtual (Cloud-Init)"
  type        = string
  sensitive   = true
}
```

Fíjate que algunos datos aparecen como *sensitive = true* para que no se muestren en pantalla o se escriban por ahí.

Habría molado (palabra rancia como yo) cargar la clave SSH y ponerla directamente en la VM de Cloud. Puedes hacerlo tu mismo:

```
variable "ssh_key_file" {
  description = "Ruta al archivo que contiene la clave pública SSH"
  type        = string
  default     = "~/.ssh/id_rsa.pub" # Opcional
}
```

Luego en la siguiente sección te digo lo que nadie te cuenta. Como asignarla.

2.7. `.tfvars`

En Terraform, tú defines variables en un archivo `variables.tf` para que tu código sea reutilizable. Sin embargo, ese archivo **solo declara que la variable existe**. El archivo `.tfvars` es el lugar donde asignas los valores reales a esas variables.

Por ejemplo declararíamos el valor de algunas variables para conectarnos mediante un token a Proxmox o el usuario y contraseña de la VM.

```
# Fichero dev.tfvars

# Credenciales de API token
pm_api_token_id = "terraform-prov@pve!terraform-token"
pm_api_token_secret = "555-el-movil-de-Sidney-Sweeny"

# Credenciales para imagen de Cloud
ci_user = "admin"
ci_password = "1234"
```

Los valores que **no** aparezcan el `.tfvars` y estén en `variables.tf` se nos preguntarán por línea de comandos.

Si me apuras yo haría distintos ficheros `.tfvars` para cada sitio donde debo desplegar. Por ejemplo tengo un `casa.tfvars` para desplegar en el Proxmox de mi casa y un `izv.tfvars` con los valores del Proxmox del IZV. O más profesionalmente un `dev.tfvars` y un `prod.tfvars`, pero yo soy poco profesional.

Y ahora lo que estabas esperando.. asignar la clave SSH en el fichero `.tfvars`:

```
sshkeys = "~/.ssh/clave_proxmox.pub"
```

y en el fichero `main.tf` usaríamos:

```
sshkeys = file(var.ssh_key_file)
```

2.7.1. Tipos de archivos de variables

Terraform busca automáticamente ciertos archivos, mientras que otros debes especificarlos tú:

`terraform.tfvars`
Terraform lo carga automáticamente si existe en el directorio.

`*.auto.tfvars`
Cualquier archivo que termine así también se carga automáticamente (útil para organizar variables por categorías, ej: `network.auto.tfvars`).

Archivos personalizados
Lo que hemos hablado antes. Si tienes uno llamado `prod.tfvars`, deberás indicarlo al ejecutar el comando:

```
terraform apply -var-file="prod.tfvars"
```

Personalmente prefiero por ahora el último método, ya que se me ocurre que puedo meter la pata y cargar cosas automáticamente que no debía. Luego, una vez que comprenda bien el problema puedo ir haciendo más ficheros para cargarse automáticamente.

2.8. `outputs.tf`

Cuando Terraform termina de desplegar una Cloud Image en Proxmox, no quieres tener que ir a la consola web para ver qué IP tiene la máquina o cuál es su ID.

Los outputs extraen esa información del estado de Terraform y la imprimen en tu terminal. También puedes verlos cuando quieras con (`terraform output`).

Te pongo un ejemplo, pero antes, fíjate que en el fichero `main.tf` que hicimos antes, definí una línea así:

```
resource "proxmox_vm_qemu" "vm_instance_1" {
```

donde indicamos el tipo de recurso (máquina virtual de qemu) y el nombre local del recurso ("vm_instance_1"). Este último nombre es el que utilizo en las salidas.

```
# Muestra la dirección IP (útil para conectar por SSH de inmediato)
output "vm_ip_address" {
  description = "La dirección IP pública/privada de la instancia"
  value       = proxmox_vm_qemu.vm_instance_1.default_ipv4_address
}

# Muestra el nombre del nodo donde se desplegó
output "proxmox_node" {
  description = "Nodo de Proxmox donde reside la VM"
  value       = proxmox_vm_qemu.vm_instance_1.target_node
}

# Muestra el ID de la VM (útil para comandos de consola 'qm')
output "vm_id" {
  description = "El ID único de la máquina virtual"
  value       = proxmox_vm_qemu.vm_instance_1.vmid
}

# Muestra un string formateado listo para copiar y pegar
output "ssh_command" {
  description = "Conectar por SSH"
  value       = "ssh m.rajoy.esfurry@${proxmox_vm_qemu.vm_instance_1.default_ipv4_address}"
}
```

3. Despliegue en Proxmox

Esta parte está en beta, ya que aunque funciona, probablemente puede mejorarse, reduciendo los permisos que se otorgan en Proxmox. Por ahora siguiendo las instrucciones del proveedor **Telmate**, es como está hecho.

En el caso de desplegar en nuestro instituto, queremos desplegar MVs en Proxmox. El administrador de Proxmox tiene que realizar las siguientes acciones:

- Crear un *rol* que otorga los permisos necesarios para desplegar en Proxmox.
- Asignar un usuario el rol anterior.

Cada usuario deberá:

- Crear un token que se utilizará para desplegar en Proxmox (con separación de privilegios desactivado).

Ahora explicaremos cada una de estas partes en detalle.

3.1. Crear un rol con permisos

Crearemos un rol llamado *TerraformProv* con permisos para desplegar en Proxmox. El rol tiene los siguientes permisos:

- Datastore.AllocateSpace
- Datastore.AllocateTemplate
- Datastore.Audit
- Pool.Allocate
- Pool.Audit
- SDN.Use
- Sys.Audit
- Sys.Console
- Sys.Modify
- VM.Allocate
- VM.Audit
- VM.Clone
- VM.Config.CDROM
- VM.Config.Cloudinit
- VM.Config.CPU
- VM.Config.Disk
- VM.Config.HWType
- VM.Config.Memory
- VM.Config.Network
- VM.Config.Options
- VM.Migrate
- VM.PowerMgmt

Mediante comando lo haríamos así:

```
pveum role add TerraformProv -privs "Datastore.AllocateSpace
Datastore.Audit Pool.Audit Pool.Allocate SDN.Use Sys.Audit Sys.Console
Sys.Modify VM.Allocate VM.Audit VM.Clone VM.Config.CDROM
VM.Config.Cloudinit VM.Config.CPU VM.Config.Disk VM.Config.HWType
VM.Config.Memory VM.Config.Network VM.Config.Options VM.Migrate
VM.PowerMgmt"
```

3.2. Asignar usuarios privilegios del rol

```
pveum aclmod / -user user@pve -role TerraformProv
```

3.3. El usuario crea un token de acceso (API token)

El usuario crea un token, **desmarcando la casilla *privilege separation***. Este token lo utilizará para desplegar con Terraform. El *token id* está formado por el nombre del usuario y por el *token name*.

Por ejemplo:

```
ciber-fraya@pve!terraform-prov
```

3.3.1. Por qué usar tokens

En general, está recomendado usar tokens por las siguientes razones

- Principio de menor privilegio
- Invalidez de la contraseña

1. El Principio de Menor Privilegio (Separación)
   Cuando usas tu contraseña, el sistema te da acceso a todo lo que tu usuario puede hacer. Con los tokens de Proxmox, puedes crear un token específico y limitarlo:
   - Sin Token: El usuario admin tiene control total.
   - Con Token: Puedes crear *admin@pve!terraform* y decirle a Proxmox que ese token solo puede gestionar máquinas virtuales, pero no ver el almacenamiento ni borrar backups, aunque el usuario "padre" sí tenga permiso para hacerlo.
2. Invalidez de la Contraseña (Aislamiento)
   Si alguien roba un API Token:

   No pueden entrar a la interfaz web
   Los tokens de Proxmox solo sirven para la API. Un atacante no podría loguearse en el panel de control (https://tu-ip:8006) con un token.

   No comprometes tu cuenta principal
   Puedes revocar (borrar) el token afectado en un segundo sin tener que cambiar la contraseña de tu usuario ni afectar a otros servicios que usen otros tokens.

4. Referencias

- https://oneuptime.com/blog/post/2026-02-23-how-to-understand-terraform-providers-and-their-role/view
- https://holori.com/terraform-providers/#:~:text=docker%20(Docker):%20This%20provider%20helps%20manage%20Docker,repositories%2C%20teams%2C%20and%20access%20permissions%20using%20Terraform.
- https://developer.hashicorp.com/terraform/language/providers
- https://developer.hashicorp.com/terraform/language
- https://developer.hashicorp.com/terraform/intro#:~:text=You%20can%20find%20all%20publicly%20available%20providers,Helm%2C%20GitHub%2C%20Splunk%2C%20DataDog%2C%20and%20many%20more.
- https://github.com/ansible/terraform-provider-ansible
- https://opentofu.org/faq/
- https://registry.terraform.io/providers/Telmate/proxmox/latest/docs

Author: Fernando Raya
Created: 2026-04-26 dom 21:21
Validate